

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Algoritma Sorting (MergeSort, QuickSort, dan ShellSort)”

DISUSUN OLEH:

NOFRI ILHAM

2511531013

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

M Galid A



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Algoritma Sorting (MergeSort, QuickSort, dan ShellSort)” ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan evaluasi dari kegiatan praktikum yang telah dilakukan.

Praktikum ini bertujuan untuk mempelajari konsep serta implementasi algoritma pengurutan data, khususnya MergeSort, QuickSort, dan ShellSort. Melalui praktikum ini, diharapkan mahasiswa dapat memahami cara kerja masing-masing algoritma, mengetahui perbedaan karakteristiknya, serta mampu mengimplementasikannya dalam pemrograman untuk menyelesaikan permasalahan pengurutan data secara efisien.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan, baik dari segi penyusunan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta semua pihak yang telah membantu dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai algoritma pengurutan data.

Padang, 28 Mei 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN	3
2.1 Pengertian Algoritma Sorting	3
2.2 Jenis-jenis Algoritma Sorting.....	3
2.2.1 Merge Sort.....	3
2.2.2 Quick Sort	4
2.2.3 Shell Sort.....	4
2.3 Implementasi pada Praktikum.....	6
2.3.1 Kode Pertama(ShellSort_2511531013)	6
2.3.2 Kode Kedua(MergeSort_2511531013).....	7
2.3.3 Kode Ketiga(QuickSort_2511531013)	9
2.3.4 Kode Keempat(BubbleSortGUI_2511531013).....	11
BAB III PENUTUP	17
3.1 Kesimpulan	17
3.2 Saran.....	17
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, proses pengurutan (sorting) merupakan salah satu teknik yang sering digunakan untuk menyusun data ke dalam urutan tertentu, baik secara menaik (ascending) maupun menurun (descending). Data yang telah terurut akan lebih mudah dikelola, dicari, dan dianalisis sehingga meningkatkan efisiensi dalam berbagai aplikasi komputer.

Terdapat berbagai algoritma sorting yang dapat digunakan untuk mengurutkan data, masing-masing memiliki cara kerja dan tingkat efisiensi yang berbeda. Oleh karena itu, penting bagi mahasiswa untuk memahami karakteristik setiap algoritma agar dapat memilih metode yang sesuai dengan kebutuhan pemrosesan data. Pada praktikum ini dipelajari tiga algoritma sorting, yaitu MergeSort, QuickSort, dan ShellSort.

MergeSort merupakan algoritma yang menggunakan metode divide and conquer dengan membagi data menjadi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut. QuickSort juga menggunakan pendekatan divide and conquer, tetapi melakukan pengurutan dengan menentukan sebuah pivot sebagai acuan pembagian data. Sementara itu, ShellSort merupakan pengembangan dari Insertion Sort yang menggunakan interval tertentu (gap) untuk mempercepat proses pengurutan.

1.2 Tujuan

Adapun tujuan dari praktikum struktur data dengan materi Algoritma Sorting adalah sebagai berikut:

1. Memahami konsep dasar algoritma sorting.
2. Mempelajari cara kerja algoritma MergeSort, QuickSort, dan ShellSort.

3. Mengimplementasikan algoritma MergeSort, QuickSort, dan ShellSort menggunakan bahasa pemrograman Java.
4. Membandingkan karakteristik dan mekanisme kerja dari algoritma MergeSort, QuickSort, dan ShellSort.
5. Meningkatkan kemampuan pemrograman dalam menerapkan algoritma pengurutan data.

1.3 Manfaat

Adapun manfaat dari praktikum struktur data dengan materi Algoritma Sorting adalah sebagai berikut:

1. Menambah pemahaman mengenai konsep dasar algoritma pengurutan (sorting) dalam pemrograman.
2. Membantu mahasiswa memahami cara kerja algoritma MergeSort, QuickSort, dan ShellSort dalam mengurutkan data.
3. Meningkatkan keterampilan dalam menganalisis proses dan hasil pengurutan data menggunakan berbagai metode sorting.
4. Memberikan wawasan mengenai perbedaan karakteristik, kelebihan, dan kekurangan dari algoritma MergeSort, QuickSort, dan ShellSort.
5. Mengembangkan kemampuan berpikir logis dan sistematis dalam menyelesaikan permasalahan pengolahan data.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Algoritma sort adalah metode atau langkah-langkah logis untuk mengurutkan sejumlah data. Data yang diurutkan dapat berupa angka, huruf, nama siswa, nilai barang, dan lain-lain.

Pengurutan dilakukan untuk:

1. memudahkan proses pencarian (searching),
2. mempercepat pengolahan data,
3. menghindari duplikasi,
4. meningkatkan performa aplikasi atau sistem.

2.2 Jenis-jenis Algoritma Sorting

2.2.1 Merge Sort

Merge Sort adalah algoritma pengurutan yang bekerja dengan cara membagi data menjadi dua bagian secara berulang hingga setiap bagian hanya berisi satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali dalam urutan yang benar.

Algoritma ini sangat stabil dan memiliki performa yang konsisten, sehingga sering digunakan dalam berbagai aplikasi yang membutuhkan keakuratan tinggi.

Kelebihan Merge Sort

1. Kompleksitas waktu stabil **$O(n \log n)$**
2. Cocok untuk dataset besar
3. Stabil (urutan elemen yang sama tetap)
4. Cocok untuk pengolahan data eksternal

Kekurangan Merge Sort

1. Membutuhkan memori tambahan
2. Implementasi lebih kompleks
3. Kurang efisien untuk dataset kecil

2.2.2 Quick Sort

Quick Sort adalah algoritma pengurutan yang bekerja dengan memilih satu elemen sebagai pivot, lalu membagi elemen lain menjadi dua bagian: yang lebih kecil dari pivot dan yang lebih besar. Proses ini dilakukan secara rekursif hingga seluruh data terurut. Quick Sort dikenal sebagai salah satu algoritma pengurutan tercepat dalam praktik.

Kelebihan Quick Sort

1. Sangat cepat dalam praktik
2. Tidak membutuhkan banyak memori tambahan
3. Cocok untuk berbagai jenis dataset
4. Efisien untuk pemrosesan internal

Kekurangan Quick Sort

1. Kompleksitas terburuk $O(n^2)$
2. Tidak stabil
3. Performa tergantung pemilihan pivot

2.2.3 Shell Sort

Algoritma Shell Sort merupakan juga dengan metode pertambahan menurun (diminishing increment). Metode ini dikembangkan oleh Donald L. Shell pada tahun 1959, sehingga sering disebut dengan Metode Shell Sort. Metode ini mengurutkan

data dengan cara membandingkan suatu data dengan data lain yang memiliki jarak tertentu, kemudian dilakukan penukaran bila diperlukan.

Kelebihan dari algoritma shellsort yaitu:

1. Algoritma ini sangat rapat dan mudah diimplementasikan.
2. Operasi pertukarannya hanya dilakukan sekali saja.
3. Waktu pengurutannya dapat lebih ditekan.
4. Mudah menggabungkannya kembali.
5. Kompleksitas selection sort relatif lebih kecil

Kekurangan dari algoritma shellsort yaitu:

1. Membutuhkan method tambahan.
2. Sulit untuk membagi masalah.

2.3 Implementasi pada Praktikum

2.3.1 Kode Pertama(ShellSort_2511531013)

```

1 package Pekan8_2511531013;
2
3 public class ShellSort_2511531013 {
4
5     public static void ShellSort_2511531013(int[] A_1013) {
6         int n_1013=A_1013.length;
7         int gap_1013=n_1013/2;
8         while (gap_1013>0) {
9             for(int i_1013=gap_1013;i_1013<n_1013;i_1013++) {
10                 int temp_1013=A_1013[i_1013];
11                 int j_1013=i_1013;
12                 while (j_1013>gap_1013&&A_1013 [j_1013-gap_1013]>temp_1013) {
13                     A_1013[j_1013]=A_1013[j_1013-gap_1013];
14                     j_1013=j_1013-gap_1013;
15                 }
16                 A_1013[j_1013]=temp_1013;
17             }
18             gap_1013=gap_1013-2;
19         }
20     }
21     public static void main(String[] args) {
22         int []data_1013= {3,10,4,6,8,9,7,2,1,5};
23         System.out.print("Sebelum: ");
24         printArray(data_1013);
25         ShellSort_2511531013(data_1013);
26         System.out.print("Sesudah (ShellSort): ");
27         printArray(data_1013);
28     }
29     private static void printArray(int[] arr_1013) {
30         for(int i_1013 :arr_1013)System.out.print(i_1013+" ");
31         System.out.println();
32     }
33 }
34
35
36
37

```

Kode 2.1

Kode diatas (ShellSort_2511531013) merupakan pengembangan dari Insertion Sort yang menggunakan konsep *gap* atau jarak tertentu dalam membandingkan elemen-elemen array. Dengan adanya *gap*, proses perpindahan elemen dapat dilakukan lebih cepat dibandingkan metode pengurutan sederhana lainnya.

Pada program ini, nilai *gap* awal ditentukan sebesar setengah dari panjang array. Selanjutnya, program melakukan proses perbandingan dan pergeseran elemen yang berjarak sesuai nilai *gap* tersebut. Setelah satu tahap selesai, nilai *gap* akan dikurangi secara bertahap hingga mencapai nilai satu, yang menandakan proses pengurutan akhir telah dilakukan.

Method main() digunakan untuk menyiapkan data yang akan diurutkan dan menampilkan isi array sebelum proses sorting dilakukan. Setelah method ShellSort_2511531013() dijalankan, data yang telah terurut ditampilkan kembali menggunakan method printArray(). Hasil tersebut menunjukkan bahwa algoritma Shell Sort berhasil mengurutkan data dari nilai terkecil hingga terbesar.

2.3.2 Kode Kedua(MergeSort_2511531013)

```

1 package Pekan8_2511531013;
2
3 public class MergeSort_2511531013 {
4     void merge(int arr[], int l_1013, int m_1013, int r_1013) {
5         // Find sizes of two subarrays to be merged
6         int n1_1013 = m_1013 - l_1013 + 1;
7         int n2_1013 = r_1013 - m_1013;
8         /* Create temp arrays */
9         int L_1013[] = new int[n1_1013];
10        int R_1013[] = new int[n2_1013];
11        /* Copy data to temp arrays */
12        for (int i_1013 = 0; i_1013 < n1_1013; ++i_1013)
13            L_1013[i_1013] = arr[l_1013 + i_1013];
14        for (int j_1013 = 0; j_1013 < n2_1013; ++j_1013)
15            R_1013[j_1013] = arr[m_1013 + 1 + j_1013];
16        int i_1013 = 0, j_1013 = 0;
17        // Initial index of merged subarray array
18        int k_1013 = l_1013;
19        while (i_1013 < n1_1013 && j_1013 < n2_1013) {
20            if (L_1013[i_1013] <= R_1013[j_1013]) {
21                arr[k_1013] = L_1013[i_1013];
22                i_1013++;
23            } else {
24                arr[k_1013] = R_1013[j_1013];
25                j_1013++;
26            }
27            k_1013++;
28        }
29        /* Copy remaining elements of L[] if any */
30        while (i_1013 < n1_1013) {
31            arr[k_1013] = L_1013[i_1013];
32            i_1013++;
33            k_1013++;
34        }
35        /* Copy remaining elements of R[] if any */
36        while (j_1013 < n2_1013) {
37            arr[k_1013] = R_1013[j_1013];
38            j_1013++;
39            k_1013++;
40        }
41    }
42    void sort(int arr_1013[], int l_1013, int r_1013) {
43        if (l_1013 < r_1013) {
44            // Find the middle point
45            int m_1013 = (l_1013 + r_1013) / 2;

```

```

46         // Sort first and second halves
47         sort(arr_1013, l_1013, m_1013);
48         sort(arr_1013, m_1013 + 1, r_1013);
49         // Merge the sorted halves
50         merge(arr_1013, l_1013, m_1013, r_1013);
51     }
52 }
53
54 /* A utility function to print array of size n */
55 static void printArray(int arr_1013[]) {
56     int n_1013 = arr_1013.length;
57     for (int i_1013 = 0; i_1013 < n_1013; ++i_1013)
58         System.out.print(arr_1013[i_1013] + " ");
59     System.out.println();
60 }
61
62 public static void main(String args[]) {
63     int arr_1013[] = { 12, 11, 13, 5, 6, 7 };
64     System.out.println("Sebelum terurut");
65     printArray(arr_1013);
66     MergeSort_2511531013 ob_1013 = new MergeSort_2511531013();
67     ob_1013.sort(arr_1013, 0, arr_1013.length - 1);
68     System.out.println("\nSesudah Terurut menggunakan merge Sort");
69     printArray(arr_1013);
70 }
71 }

```

Kode 2.2

Kode diatas (MergeSort_2511531013) digunakan untuk mengimplementasikan algoritma Merge Sort dalam proses pengurutan data integer secara ascending. Algoritma ini menggunakan metode *divide and conquer*, yaitu membagi array menjadi beberapa bagian yang lebih kecil hingga setiap bagian hanya memiliki satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali secara bertahap dalam keadaan terurut.

Proses pembagian array dilakukan oleh method sort(), yang bekerja secara rekursif untuk membagi data menjadi dua subarray. Setelah proses pembagian selesai, method merge() digunakan untuk menggabungkan dua subarray yang telah terurut menjadi satu array yang lebih besar dengan urutan yang benar. Pada tahap ini dilakukan perbandingan elemen dari kedua subarray untuk menentukan posisi setiap elemen.

Pada method main(), program menyediakan data awal berupa array integer yang kemudian ditampilkan sebelum proses pengurutan dilakukan. Setelah method sort() dijalankan, hasil

pengurutan ditampilkan kembali menggunakan method `printArray()`. Dengan demikian, pengguna dapat melihat perbedaan data sebelum dan sesudah proses pengurutan menggunakan algoritma Merge Sort.

2.3.3 Kode Ketiga(QuickSort_2511531013)

```
package Pekan8_2511531013;

public class QuickSort_2511531013 {

    static void swap(int[] arr_1013, int i_1013, int j_1013) {
        int temp_1013 = arr_1013[i_1013];
        arr_1013[i_1013] = arr_1013[j_1013];
        arr_1013[j_1013] = temp_1013;
    }

    static void medianOfThree(int[] arr_1013, int low_1013, int high_1013) {
        int mid_1013 = low_1013 + (high_1013 - low_1013) / 2;

        if (arr_1013[low_1013] > arr_1013[mid_1013]) {
            swap(arr_1013, low_1013, mid_1013);
        }

        if (arr_1013[low_1013] > arr_1013[high_1013]) {
            swap(arr_1013, low_1013, high_1013);
        }

        if (arr_1013[mid_1013] > arr_1013[high_1013]) {
            swap(arr_1013, mid_1013, high_1013);
        }

        swap(arr_1013, mid_1013, high_1013);
    }

    static int partition(int[] arr_1013, int low_1013, int high_1013) {
        medianOfThree(arr_1013, low_1013, high_1013);

        int pivot_1013 = arr_1013[high_1013];
        int i_1013 = low_1013 - 1;

        for (int j_1013 = low_1013; j_1013 <= high_1013 - 1; j_1013++) {
            if (arr_1013[j_1013] < pivot_1013) {
                i_1013++;
                swap(arr_1013, i_1013, j_1013);
            }
        }
        swap(arr_1013, i_1013 + 1, high_1013);
        return i_1013 + 1;
    }

    static void quickSort(int[] arr_1013, int low_1013, int high_1013) {
```

```

46     if (low_1013 < high_1013) {
47         int pi_1013 = partition(arr_1013, low_1013, high_1013);
48
49         quickSort(arr_1013, low_1013, pi_1013 - 1);
50         quickSort(arr_1013, pi_1013 + 1, high_1013);
51     }
52 }
53 public static void printArr(int[] arr_1013) {
54     for (int i_1013 = 0; i_1013 < arr_1013.length; i_1013++) {
55         System.out.print(arr_1013[i_1013] + " ");
56     }
57     System.out.println();
58 }
59 public static void main(String[] args) {
60     int[] arr_1013 = {10, 7, 8, 9, 1, 5};
61     int N_1013 = arr_1013.length;
62
63     System.out.print("Data sebelum diurutkan: ");
64     printArr(arr_1013);
65
66     quickSort(arr_1013, 0, N_1013 - 1);
67
68     System.out.print("Data terurut quicksort: ");
69     printArr(arr_1013);
70 }
71 }

```

Kode 2.3

Kode diatas (QuickSort_2511531013) digunakan untuk mengurutkan data integer menggunakan algoritma Quick Sort. Algoritma ini bekerja dengan memilih sebuah elemen sebagai *pivot* yang digunakan sebagai acuan untuk membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Setelah proses pembagian selesai, masing-masing bagian akan diurutkan kembali secara rekursif.

Pada program ini, pemilihan pivot dilakukan menggunakan metode *Median of Three* yang bertujuan untuk memperoleh pivot yang lebih representatif sehingga performa algoritma menjadi lebih baik. Method `partition()` berfungsi untuk mengatur posisi elemen berdasarkan nilai pivot, sedangkan method `quickSort()` digunakan untuk menjalankan proses pengurutan secara rekursif hingga seluruh data berada dalam urutan yang benar.

Dalam method `main()`, program menyiapkan data berupa array integer yang akan diurutkan. Data ditampilkan sebelum proses pengurutan, kemudian algoritma Quick Sort dijalankan. Setelah seluruh proses selesai, hasil pengurutan ditampilkan kembali

sehingga pengguna dapat melihat perubahan susunan data yang terjadi setelah menggunakan algoritma Quick Sort.

2.3.4 Kode Keempat(BubleSortGUI_2511531013)

```

1 package Pekan8_2511531013;
2
3 import javax.swing.*;
4
5
6
7
8 public class BubleSortGUI_2511531013 extends JFrame {
9
10     private JTextField inputField_1013;
11     private JButton setButton_1013;
12     private JButton stepButton_1013;
13     private JButton resetButton_1013;
14     private JTextArea stepArea_1013;
15     private JPanel panelArray_1013;
16
17     private int[] array_1013;
18     private JLabel[] labelArray_1013;
19
20     private int i_1013 = 0;
21     private int j_1013 = 0;
22     private int stepCount_1013 = 1;
23     private boolean sorting_1013 = false;
24
25     public BubleSortGUI_2511531013() {
26         setTitle("Visualisasi Bubble Sort");
27         setSize(800, 500);
28         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29         setLocationRelativeTo(null);
30         setLayout(new BorderLayout(10, 10));
31
32         // Panel input bagian atas
33         JPanel inputPanel = new JPanel(new FlowLayout());
34
35         JLabel inputLabel = new JLabel("Masukkan angka (pisahkan dengan koma):");
36         inputField_1013 = new JTextField(30);
37
38         setButton_1013 = new JButton("Set Array");
39         stepButton_1013 = new JButton("Step");
40         resetButton_1013 = new JButton("Reset");
41
42         stepButton_1013.setEnabled(false);
43
44         inputPanel.add(inputLabel);
45         inputPanel.add(inputField_1013);
46         inputPanel.add(setButton_1013);
47         inputPanel.add(stepButton_1013);

```

```

48     inputPanel.add(resetButton_1013);
49
50     add(inputPanel, BorderLayout.NORTH);
51
52     // Panel untuk menampilkan array
53     panelArray_1013 = new JPanel(new FlowLayout());
54     panelArray_1013.setPreferredSize(new Dimension(750, 120));
55     add(panelArray_1013, BorderLayout.CENTER);
56
57     // Text area untuk menampilkan langkah sorting
58     stepArea_1013 = new JTextArea();
59     stepArea_1013.setEditable(false);
60     stepArea_1013.setFont(new Font("Monospaced", Font.PLAIN, 14));
61
62     JScrollPane scrollPane = new JScrollPane(stepArea_1013);
63     scrollPane.setPreferredSize(new Dimension(750, 250));
64
65     add(scrollPane, BorderLayout.SOUTH);
66
67     // Event tombol
68     setButton_1013.addActionListener(new ActionListener() {
69         @Override
70         public void actionPerformed(ActionEvent e) {
71             setArrayFromInput();
72         }
73     });
74
75     stepButton_1013.addActionListener(new ActionListener() {
76         @Override
77         public void actionPerformed(ActionEvent e) {
78             performStep();
79         }
80     });
81
82     resetButton_1013.addActionListener(new ActionListener() {
83         @Override
84         public void actionPerformed(ActionEvent e) {
85             reset();
86         }
87     });
88 }
89
90 private void setArrayFromInput() {

```

```

91 String text = inputField_1013.getText().trim();
92
93 if (text.isEmpty()) {
94     return;
95 }
96
97 String[] parts_1013 = text.split(",");
98 array_1013 = new int[parts_1013.length];
99
100 try {
101     for (int k = 0; k < parts_1013.length; k++) {
102         array_1013[k] = Integer.parseInt(parts_1013[k].trim());
103     }
104 } catch (NumberFormatException e) {
105     JOptionPane.showMessageDialog(
106         this,
107         "Masukkan hanya angka yang dipisahkan koma!",
108         "Error",
109         JOptionPane.ERROR_MESSAGE
110     );
111     return;
112 }
113
114 i_1013 = 0;
115 j_1013 = 0;
116 stepCount_1013 = 1;
117 sorting_1013 = true;
118
119 stepButton_1013.setEnabled(true);
120 stepArea_1013.setText("");
121 panelArray_1013.removeAll();
122
123 labelArray_1013 = new JLabel[array_1013.length];
124
125 for (int k = 0; k < array_1013.length; k++) {
126     labelArray_1013[k] = new JLabel(String.valueOf(array_1013[k]));
127     labelArray_1013[k].setFont(new Font("Arial", Font.BOLD, 24));
128     labelArray_1013[k].setOpaque(true);
129     labelArray_1013[k].setBackground(Color.WHITE);
130     labelArray_1013[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
131     labelArray_1013[k].setPreferredSize(new Dimension(50, 50));
132     labelArray_1013[k].setHorizontalAlignment(SwingConstants.CENTER);
133
134     panelArray_1013.add(labelArray_1013[k]);
135 }
136
137 panelArray_1013.revalidate();
138 panelArray_1013.repaint();
139 }
140
141 private void performStep() {
142     if (!sorting_1013 || i_1013 >= array_1013.length - 1) {
143         sorting_1013 = false;
144         stepButton_1013.setEnabled(false);
145         JOptionPane.showMessageDialog(this, "Sorting selesai!");
146         return;
147     }
148
149     resetHighlights();
150
151     StringBuilder stepLog = new StringBuilder();
152
153     labelArray_1013[j_1013].setBackground(Color.CYAN);
154     labelArray_1013[j_1013 + 1].setBackground(Color.CYAN);
155
156     if (array_1013[j_1013] > array_1013[j_1013 + 1]) {
157         // Swap
158         int temp = array_1013[j_1013];
159         array_1013[j_1013] = array_1013[j_1013 + 1];
160         array_1013[j_1013 + 1] = temp;
161
162         labelArray_1013[j_1013].setBackground(Color.RED);
163         labelArray_1013[j_1013 + 1].setBackground(Color.RED);
164
165         stepLog.append("Langkah ").append(stepCount_1013).append(": ")
166             .append("Menukar elemen ke-").append(j_1013)
167             .append(" (").append(array_1013[j_1013 + 1]).append(")")
168             .append(" dengan ke-").append(j_1013 + 1)
169             .append(" (").append(array_1013[j_1013]).append(")\n");
170
171     } else {
172         stepLog.append("Langkah ").append(stepCount_1013).append(": ")
173             .append("Tidak ada pertukaran antara ke-")
174             .append(j_1013).append(" dan ke-")
175             .append(j_1013 + 1).append("\n");
176     }
177

```



```

178         stepLog.append("Hasil: ").append(arrayToString(array_1013)).append("\n\n");
179         stepArea_1013.append(stepLog.toString());
180
181         updateLabels();
182
183         j_1013++;
184
185     if (j_1013 >= array_1013.length - 1 - i_1013) {
186         j_1013 = 0;
187         i_1013++;
188     }
189
190     stepCount_1013++;
191
192     if (i_1013 >= array_1013.length - 1) {
193         sorting_1013 = false;
194         stepButton_1013.setEnabled(false);
195         resetHighlights();
196
197         for (JLabel label : labelArray_1013) {
198             label.setBackground(Color.GREEN);
199         }
200
201         JOptionPane.showMessageDialog(this, "Sorting selesai!");
202     }
203 }
204
205 private void updateLabels() {
206     for (int k_1013 = 0; k_1013 < array_1013.length; k_1013++) {
207         labelArray_1013[k_1013].setText(String.valueOf(array_1013[k_1013]));
208     }
209 }
210
211 private void resetHighlights() {
212     for (JLabel label : labelArray_1013) {
213         label.setBackground(Color.WHITE);
214     }
215 }
216
217 private void reset() {
218     inputField_1013.setText("");
219     panelArray_1013.removeAll();

```

```

220     panelArray_1013.revalidate();
221     panelArray_1013.repaint();
222
223     stepArea_1013.setText("");
224     stepButton_1013.setEnabled(false);
225
226     sorting_1013 = false;
227     i_1013 = 0;
228     j_1013 = 0;
229     stepCount_1013 = 1;
230 }
231
232 private String arrayToString(int[] arr) {
233     StringBuilder sb = new StringBuilder();
234
235     for (int k = 0; k < arr.length; k++) {
236         sb.append(arr[k]);
237
238         if (k < arr.length - 1) {
239             sb.append(", ");
240         }
241     }
242
243     return sb.toString();
244 }
245
246 public static void main(String[] args) {
247     SwingUtilities.invokeLater(new Runnable() {
248         @Override
249         public void run() {
250             new BubleSortGUI_2511531013().setVisible(true);
251         }
252     });
253 }
254 }

```

Kode 2.4

Kode diatas (BubleSortGUI_2511531013) merupakan aplikasi berbasis Java Swing yang dirancang untuk memvisualisasikan proses kerja algoritma Bubble Sort secara interaktif. Program menyediakan antarmuka grafis yang memungkinkan pengguna memasukkan sejumlah data angka melalui sebuah kolom input. Data yang dimasukkan kemudian ditampilkan dalam bentuk label sehingga lebih mudah diamati selama proses pengurutan berlangsung.

Setelah data dimasukkan, pengguna dapat menjalankan proses pengurutan secara bertahap menggunakan tombol *Step*. Pada setiap langkah, program akan membandingkan dua elemen yang berdekatan dan melakukan pertukaran apabila elemen pertama memiliki nilai yang lebih besar dari elemen kedua. Proses

perbandingan dan pertukaran ditampilkan menggunakan perubahan warna pada elemen array sehingga pengguna dapat melihat secara langsung jalannya algoritma Bubble Sort.

Selain menampilkan perubahan posisi data, program juga menyediakan area teks yang berisi informasi setiap langkah pengurutan yang dilakukan. Fitur ini membantu pengguna memahami proses Bubble Sort secara lebih detail. Setelah seluruh data berhasil diurutkan, program akan menampilkan hasil akhir dan memberikan notifikasi bahwa proses sorting telah selesai dilakukan.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting memiliki peran penting dalam proses pengolahan data karena dapat menyusun data secara terurut sehingga lebih mudah untuk dikelola dan dicari. Pada praktikum ini telah dipelajari tiga algoritma sorting, yaitu Merge Sort, Quick Sort, dan Shell Sort yang masing-masing memiliki cara kerja serta karakteristik yang berbeda.

Melalui implementasi program menggunakan bahasa Java, dapat dipahami bahwa Merge Sort bekerja dengan metode *divide and conquer*, Quick Sort menggunakan pivot sebagai acuan pengurutan, sedangkan Shell Sort memanfaatkan konsep *gap* untuk mempercepat proses pengurutan. Ketiga algoritma tersebut berhasil mengurutkan data sesuai dengan tujuan yang diharapkan.

Selain memahami konsep sorting, praktikum ini juga memberikan pengalaman dalam menulis dan mengembangkan program sesuai ketentuan yang diberikan. Dari kegiatan ini diperoleh pemahaman bahwa ketelitian dalam penulisan kode, termasuk dalam penamaan variabel dan method, sangat berpengaruh terhadap keterbacaan serta proses pengembangan program.

3.2 Saran

Praktikum selanjutnya diharapkan dapat memberikan lebih banyak variasi studi kasus sehingga mahasiswa dapat memahami penerapan algoritma sorting pada permasalahan yang lebih kompleks. Selain itu, akan lebih baik apabila dilakukan analisis perbandingan performa antar algoritma menggunakan jumlah data yang lebih besar agar perbedaan efisiensinya dapat diamati secara lebih nyata.

Dalam pelaksanaan praktikum, ketentuan penulisan program yang diberikan sebaiknya tetap mempertimbangkan aspek keterbacaan kode sehingga mahasiswa dapat lebih fokus dalam memahami logika algoritma yang dipelajari. Dengan demikian, tujuan pembelajaran dapat tercapai secara lebih optima

DAFTAR PUSTAKA

- [1] “Algoritma Sort: Pengertian, Jenis, Cara Kerja dan Contoh Lengkap”, [Daring]. Tersedia pada: <https://terapan-ti.vokasi.unesa.ac.id/post/algoritma-sort-pengertian-jenis-cara-kerja-dan-contoh-lengkap>. [Diakses: 31-Mei-2026].
- [2] Heru, “Implementasi Algoritma Sorting pada Pengolahan Data,” Jurnal Sistem Informasi dan Komputer (JUSIKOM), [PDF]. Tersedia pada: file PDF yang digunakan dalam praktikum. [Diakses: 31-Mei-2026].
- [3] “Sorting Lanjutan: Merge Sort dan Quick Sort” , [Daring]. Tersedia pada: <https://jakarta.telkomuniversity.ac.id/sorting-lanjutan-merge-sort-dan-quick-sort/>. [Diakses: 31-Mei-2026].

LAPORAN TUGAS PEKAN 8

STRUKTUR DATA

DISUSUN OLEH:

NOFRI ILHAM

2511531013

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. dataLagu_2511531013

```
1 package Pekan8_2511531013;
2
3 public class dataLagu_2511531013 {
4
5     private String judul_1013;
6     private String penyanyi_1013;
7     private int durasi_1013;
8
9     public dataLagu_2511531013(
10         String judul_1013,
11         String penyanyi_1013,
12         int durasi_1013) {
13
14         this.judul_1013 = judul_1013;
15         this.penyanyi_1013 = penyanyi_1013;
16         this.durasi_1013 = durasi_1013;
17     }
18     public String getJudul_1013() {
19         return judul_1013;
20     }
21     public String getPenyanyi_1013() {
22         return penyanyi_1013;
23     }
24     public int getDurasi_1013() {
25         return durasi_1013;
26     }
27     @Override
28     public String toString() {
29         return judul_1013 + " - " +
30             penyanyi_1013 + " - " +
31             durasi_1013 + " detik";
32     }
33 }
```

Kode di atas merupakan kelas `dataLagu_2511531013` yang digunakan untuk menyimpan data lagu dalam playlist. Pada kelas ini terdapat tiga atribut utama yaitu `judul_1013`, `penyanyi_1013`, dan `durasi_1013`. Atribut tersebut digunakan untuk menyimpan informasi mengenai judul lagu, nama penyanyi, dan durasi lagu dalam satuan detik. Dengan adanya kelas ini, setiap lagu dapat direpresentasikan sebagai sebuah objek yang memiliki data tersendiri.

Selain atribut, kelas ini juga memiliki constructor `dataLagu_2511531013()` yang berfungsi untuk menginisialisasi data lagu

ketika objek dibuat. Terdapat pula beberapa method getter seperti `getJudul_1013()`, `getPenyanyi_1013()`, dan `getDurasi_1013()` yang digunakan untuk mengambil nilai dari masing-masing atribut. Penggunaan method getter membantu menjaga keamanan data karena atribut tidak diakses secara langsung dari luar kelas.

Pada bagian akhir terdapat method `toString()` yang digunakan untuk menampilkan informasi lagu dalam bentuk teks yang lebih mudah dibaca. Method ini akan menggabungkan judul lagu, nama penyanyi, dan durasi lagu menjadi satu kalimat sehingga saat objek ditampilkan menggunakan `System.out.println()`, informasi lagu dapat langsung terlihat dengan format yang rapi.

2. Sorting_2511531013

```
1 package Pekan8_2511531013;
2 import java.util.Scanner;
3
4 public class Sorting_2511531013 {
5
6     Scanner input_1013 = new Scanner(System.in);
7
8     // Array untuk menyimpan maksimal 20 data lagu
9     static dataLagu_2511531013[] dataLagu_1013 =
10         new dataLagu_2511531013[20];
11
12     // Method untuk menginput data lagu
13     public static dataLagu_2511531013 inputData_1013(
14         Scanner input_1013) {
15
16         System.out.print("Judul Lagu : ");
17         String judul_1013 = input_1013.nextLine();
18
19         System.out.print("Penyanyi Lagu : ");
20         String penyanyi_1013 = input_1013.nextLine();
21
22         System.out.print("Durasi Lagu : ");
23         int durasi_1013 = input_1013.nextInt();
24
25         input_1013.nextLine();
26
27         return new dataLagu_2511531013(
28             judul_1013,
29             penyanyi_1013,
30             durasi_1013
31         );
32     }
33
34     // Menukar posisi dua data lagu
35     static void swap_1013(
36         dataLagu_2511531013[] dataLagu_1013,
37         int i_1013,
38         int j_1013) {
39
40         dataLagu_2511531013 temp_1013 =
41             dataLagu_1013[i_1013];
42
43         dataLagu_1013[i_1013] =
44             dataLagu_1013[j_1013];
45
46         dataLagu_1013[j_1013] =
47             temp_1013;
48     }
49 }
```

```

        dataLagu_1013[j_1013] =
            temp_1013;
    }

    // Mengatur pivot menggunakan metode Median of Three
    static void medianOfThree_1013(
        dataLagu_2511531013[] dataLagu_1013,
        int low_1013,
        int high_1013) {

        int mid_1013 =
            low_1013 + (high_1013 - low_1013) / 2;

        if (dataLagu_1013[low_1013].getDurasi_1013() >
            dataLagu_1013[mid_1013].getDurasi_1013()) {

            swap_1013(dataLagu_1013,
                low_1013,
                mid_1013);
        }

        if (dataLagu_1013[low_1013].getDurasi_1013() >
            dataLagu_1013[high_1013].getDurasi_1013()) {

            swap_1013(dataLagu_1013,
                low_1013,
                high_1013);
        }

        if (dataLagu_1013[mid_1013].getDurasi_1013() >
            dataLagu_1013[high_1013].getDurasi_1013()) {

            swap_1013(dataLagu_1013,
                mid_1013,
                high_1013);
        }

        swap_1013(dataLagu_1013,
            mid_1013,
            high_1013);
    }

    // Membagi data berdasarkan pivot
    static int partition_1013(
        dataLagu_2511531013[] dataLagu_1013,

```

```

91         int low_1013,
92         int high_1013) {
93
94         medianOfThree_1013(
95             dataLagu_1013,
96             low_1013,
97             high_1013);
98
99         dataLagu_2511531013 pivot_1013 =
100             dataLagu_1013[high_1013];
101
102         int i_1013 = low_1013 - 1;
103
104         for (int j_1013 = low_1013;
105             j_1013 <= high_1013 - 1;
106             j_1013++) {
107
108             if (dataLagu_1013[j_1013]
109                 .getDurasi_1013() <
110                 pivot_1013.getDurasi_1013()) {
111
112                 i_1013++;
113
114                 swap_1013(
115                     dataLagu_1013,
116                     i_1013,
117                     j_1013);
118             }
119         }
120
121         swap_1013(
122             dataLagu_1013,
123             i_1013 + 1,
124             high_1013);
125
126         return i_1013 + 1;
127     }
128
129     // Proses pengurutan menggunakan Quick Sort
130     static void quickSort_1013(
131         dataLagu_2511531013[] dataLagu_1013,
132         int low_1013,
133         int high_1013) {
134
135         if (low_1013 < high_1013) {

```

```

        int pi_1013 =
            partition_1013(
                dataLagu_1013,
                low_1013,
                high_1013);

        quickSort_1013(
            dataLagu_1013,
            low_1013,
            pi_1013 - 1);

        quickSort_1013(
            dataLagu_1013,
            pi_1013 + 1,
            high_1013);
    }
}

// Menampilkan data lagu
public static void tampilData_1013(
    dataLagu_2511531013[] dataLagu_1013,
    int jumlah_1013) {

    for (int i_1013 = 0;
        i_1013 < jumlah_1013;
        i_1013++) {

        System.out.println(
            (i_1013 + 1) + ". "
            + dataLagu_1013[i_1013]);
    }
}

// Program utama
public static void main(String[] args) {

    Scanner input_1013 =
        new Scanner(System.in);

    int jumlah_1013;

    System.out.println(
        "=== Sorting Playlist NIM: 2511531013 ===");

do {

    System.out.print(
        "Jumlah Lagu (Minimal 7): ");

    jumlah_1013 =
        input_1013.nextInt();

    input_1013.nextLine();

    if (jumlah_1013 < 7) {

        System.out.println(
            "Minimal harus 7 lagu!");
    }

} while (jumlah_1013 < 7);

for (int i_1013 = 0;
    i_1013 < jumlah_1013;
    i_1013++) {

    System.out.println(
        "\nData Lagu Ke-" +
        (i_1013 + 1));

    dataLagu_1013[i_1013] =
        inputData_1013(
            input_1013);
    }

    System.out.println(
        "\nData Sebelum Sorting:");

    tampilData_1013(
        dataLagu_1013,
        jumlah_1013);

    quickSort_1013(
        dataLagu_1013,
        0,
        jumlah_1013 - 1);

    System.out.println(
        "\nData Setelah Quick Sort (Durasi Asc):");

    tampilData_1013(
        dataLagu_1013,
        jumlah_1013);
}
}

```

Kode di atas merupakan program utama yang digunakan untuk mengelola dan mengurutkan data playlist lagu menggunakan algoritma

Quick Sort. Program menyediakan array `dataLagu_1013` yang mampu menyimpan maksimal 20 data lagu. Selain itu, terdapat method `inputData_1013()` yang berfungsi untuk menerima input judul lagu, nama penyanyi, dan durasi lagu dari pengguna kemudian menyimpannya ke dalam objek `dataLagu_2511531013`.

Proses pengurutan dilakukan menggunakan algoritma Quick Sort melalui method `quickSort_1013()`. Dalam prosesnya, program memanfaatkan method `partition_1013()` untuk membagi data berdasarkan pivot dan method `swap_1013()` untuk menukar posisi dua data lagu yang belum berada pada urutan yang benar. Untuk memperoleh pivot yang lebih baik, digunakan metode *Median of Three* pada method `medianOfThree_1013()`, sehingga proses pengurutan dapat berjalan lebih efisien dan menghasilkan urutan durasi lagu dari yang terkecil hingga terbesar.

Pada method `main()`, pengguna diminta memasukkan jumlah lagu dengan ketentuan minimal tujuh lagu sesuai instruksi tugas. Setelah seluruh data lagu dimasukkan, program akan menampilkan data sebelum proses sorting menggunakan method `tampilData_1013()`. Selanjutnya data diurutkan menggunakan Quick Sort berdasarkan durasi lagu secara ascending, kemudian hasil pengurutan ditampilkan kembali sehingga pengguna dapat melihat perbedaan antara data sebelum dan sesudah proses sorting dilakukan.

3. Alasan Memilih Algoritma Quick Sort

Saya memilih algoritma Quick Sort karena menurut saya algoritma ini cukup efektif dan mudah dipahami untuk mengurutkan data. Quick Sort bekerja dengan membagi data berdasarkan sebuah pivot, kemudian mengurutkan data yang lebih kecil dan lebih besar secara terpisah hingga seluruh data tersusun dengan benar. Cara kerja tersebut membuat proses pengurutan menjadi lebih cepat dibandingkan beberapa algoritma sorting sederhana.

Selain itu, Quick Sort memiliki performa yang baik dengan kompleksitas waktu rata-rata $O(n \log n)$ sehingga cocok digunakan untuk mengurutkan data dalam jumlah yang cukup banyak. Pada tugas ini, algoritma Quick Sort digunakan untuk mengurutkan playlist lagu berdasarkan durasi dari yang terpendek hingga yang terpanjang. Saya juga memilih algoritma ini karena implementasinya mirip dengan materi yang telah dipelajari pada praktikum sehingga lebih mudah untuk dipahami dan diterapkan pada program yang dibuat.

4. Output

```
=== Sorting Playlist NIM: 2511531013 ===
Jumlah Lagu (Minimal 7): 7

Data Lagu Ke-1
Judul Lagu   : ghost
Penyanyi Lagu : nofri
Durasi Lagu  : 272

Data Lagu Ke-2
Judul Lagu   : hold on
Penyanyi Lagu : ilham
Durasi Lagu  : 250

Data Lagu Ke-3
Judul Lagu   : beauty and the beast
Penyanyi Lagu : justin bieber
Durasi Lagu  : 300

Data Lagu Ke-4
Judul Lagu   : switch on
Penyanyi Lagu : charie puth
Durasi Lagu  : 278

Data Lagu Ke-5
Judul Lagu   : rasa ini
Penyanyi Lagu : viera
Durasi Lagu  : 248

Data Lagu Ke-6
Judul Lagu   : be with you
Penyanyi Lagu : akon
Durasi Lagu  : 270

Data Lagu Ke-7
Judul Lagu   : night changes
Penyanyi Lagu : one direction
Durasi Lagu  : 295
```

```
Data Sebelum Sorting:
1. ghost - nofri - 272 detik
2. hold on - ilham - 250 detik
3. beauty and the beast - justin bieber - 300 detik
4. switch on - charie puth - 278 detik
5. rasa ini - viera - 248 detik
6. be with you - akon - 270 detik
7. night changes - one direction - 295 detik
```

```
Data Setelah Quick Sort (Durasi Asc):
1. rasa ini - viera - 248 detik
2. hold on - ilham - 250 detik
3. be with you - akon - 270 detik
4. ghost - nofri - 272 detik
5. switch on - charie puth - 278 detik
6. night changes - one direction - 295 detik
7. beauty and the beast - justin bieber - 300 detik
```